

Object Tracking

This tutorial is an advanced mode for color block positioning experiment, adding robotic arm following functionality.

Overview: Object tracking includes three modes - color tracking, color learning tracking, and face tracking. The principle is to perform certain processing on images through the camera, identify the target in a specific way, obtain the coordinate position of the target under the camera, calculate the deviation between the target center point and the image center point, and through PID algorithm debugging, drive the robotic arm to move. This makes the target center point coincide with the image center point.

Source code path:

```
dofbot_ws/src/dofbot_color_follow/Color Tracking Color Follow.ipynb
```

Control Design

- Import header files

```
import cv2 as cv
import threading
import random
from time import sleep
import ipywidgets as widgets
from IPython.display import display
from color_follow import color_follow
```

- Initialize robotic arm position

```
import Arm_Lib
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 135, 20, 25, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

- Create instance, initialize parameters

```
follow = color_follow()
# Initialize mode
model = 'General'
# Initialize HSV_learning value
HSV_learning = ()
# Initialize HSV values
color_hsv = {"red" : ((0, 43, 46), (10, 255, 255)),
             "green" : ((35, 43, 46), (77, 255, 255)),
             "blue" : ((100, 43, 46), (124, 255, 255)),
             "yellow": ((26, 43, 46), (34, 255, 255))}
# Set random colors
color = [[random.randint(0, 255) for _ in range(3)] for _ in range(255)]
```

- Create controls

```

button_layout = widgets.Layout(width='200px',
height='100px',align_self='center')
# Output widget
output = widgets.Output()
# Color tracking
color_follow = widgets.Button(description='color_follow', button_style='success',
layout=button_layout)
# Select color
choose_color = widgets.ToggleButtons(options=['red', 'green', 'blue', 'yellow'],
button_style='success',tooltips=['Description of slow', 'Description of regular',
'Description of fast'])
# Cancel tracking
follow_cancel = widgets.Button(description='follow_cancel',
button_style='danger', layout=button_layout)
# Learning color
learning_color = widgets.Button(description='learning_color',
button_style='primary', layout=button_layout)
# Learning color tracking
learning_follow = widgets.Button(description='learning_follow',
button_style='success', layout=button_layout)
# Exit
exit_button = widgets.Button(description='Exit', button_style='danger',
layout=button_layout)
# Image widget
imgbox = widgets.Image(format='jpg', height=480, width=640,
layout=widgets.Layout(align_self='auto'))
# Vertical layout
img_box = widgets.VBox([imgbox, choose_color],
layout=widgets.Layout(align_self='auto'))
# Vertical layout
slider_box = widgets.VBox([color_follow, learning_color,
learning_follow,follow_cancel,exit_button],layout=widgets.Layout(align_self='auto'))
# Horizontal layout
controls_box = widgets.HBox([img_box, slider_box],
layout=widgets.Layout(align_self='auto'))
# ['auto', 'flex-start', 'flex-end', 'center', 'baseline', 'stretch', 'inherit',
'initial', 'unset']

```

- Mode switching

```

def color_follow_Callback(value):
    global model
    model = 'color_follow'

def learning_color_Callback(value):
    global model
    model = 'learning_color'

def learning_follow_Callback(value):
    global model
    model = 'learning_follow'

def follow_cancel_Callback(value):
    global model
    model = 'General'

```

```
def exit_button_Callback(value):
    global model
    model = 'Exit'
color_follow.on_click(color_follow_Callback)
learning_color.on_click(learning_color_Callback)
learning_follow.on_click(learning_follow_Callback)
follow_cancel.on_click(follow_cancel_Callback)
exit_button.on_click(exit_button_Callback)
```

- Main program

```
def camera():
    global HSV_learning,model
    # Open camera
    capture = cv.VideoCapture(0)
    capture.set(3, 640)
    capture.set(4, 480)
    capture.set(5, 30) #Set frame rate
    # Loop when camera is opened normally
    while capture.isOpened():
        try:
            # Read each frame of the camera
            _, img = capture.read()
            # Uniform image size
            img = cv.resize(img, (640, 480))
            if model == 'color_follow':
                img = follow.follow_function(img,color_hsv[choose_color.value])
                # Add text

                cv.putText(img, choose_color.value, (int(img.shape[0] / 2), 50),
cv.FONT_HERSHEY_SIMPLEX, 2, color[random.randint(0, 254)], 2)
            if model == 'learning_color':
                img,HSV_learning = follow.get_hsv(img)
            if model == 'learning_follow':
                img = follow.learning_follow(img, HSV_learning)
                # Add text
                cv.putText(img,'LeColor', (200, 50), cv.FONT_HERSHEY_SIMPLEX, 1,
color[random.randint(0, 254)], 1)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break

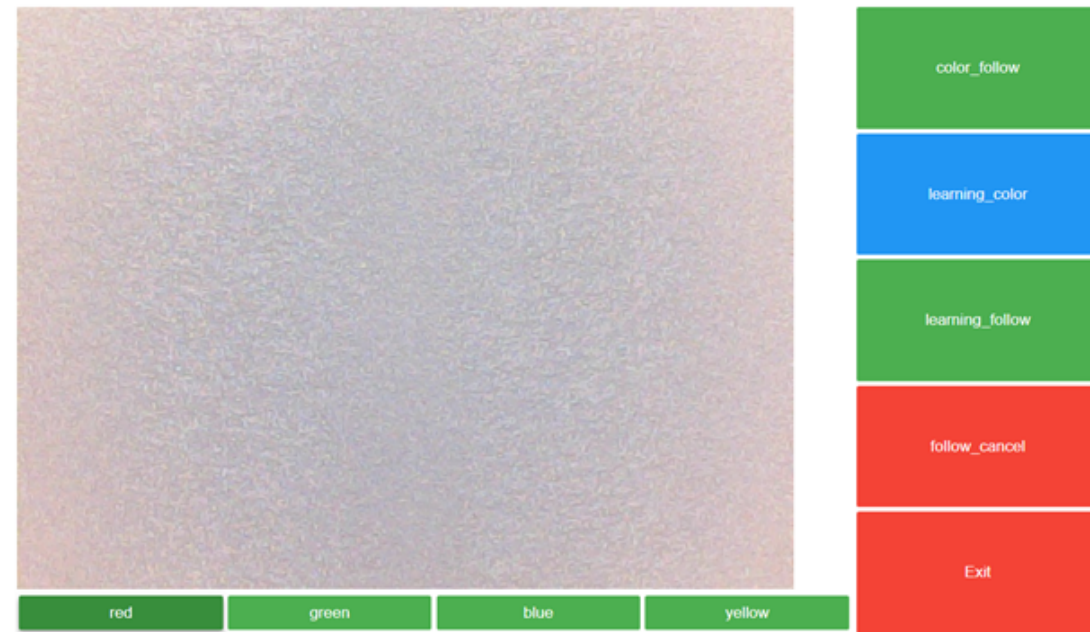
            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt:capture.release()
```

- Start

```
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

The PID control algorithm adjustment was covered in the PID Algorithm section. For adjustment, please refer to PID Algorithm for learning. For the color tracking and color learning tracking library code, please see `dofbot_ws/src/dofbot_color_follow/color_follow.py`

Color Tracking

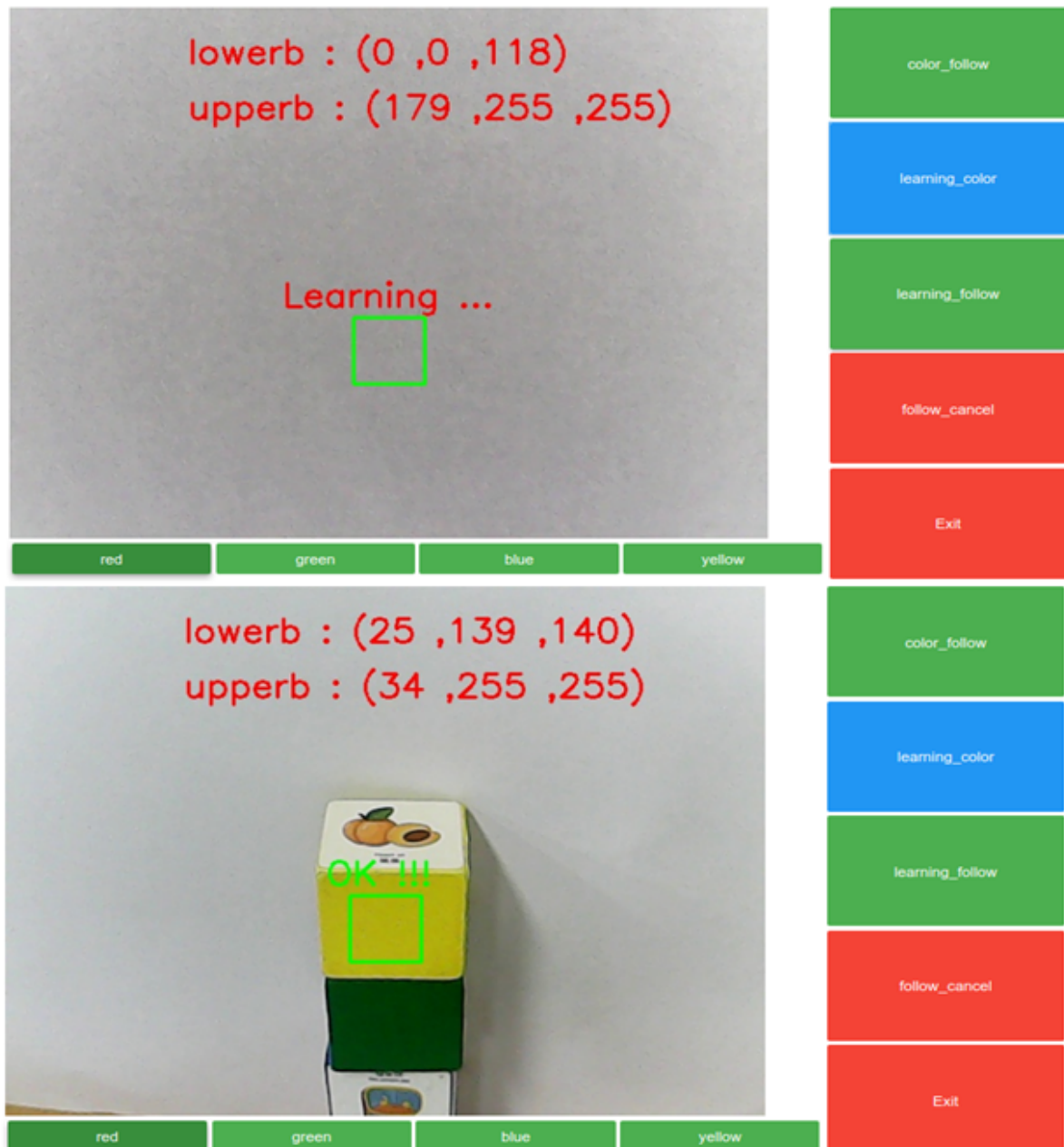


By default, tracking is not enabled. You need to select the function button on the right. In color tracking mode,

Start the code block, click the [color_follow] button to start color tracking. Color can be switched at the bottom of the image. Place a block in the field of view. When the camera recognizes it, the robotic arm tracks the block movement, aligning with the center of the block.

Click the [follow_cancel] button to cancel tracking without exiting the program. Click the Exit button to exit the program.

Color Learning Tracking



(1) Start the code block, click the [learning_color] button, and a box will appear in the center of the screen, as shown in the left image.

(2) Place an object inside the box, and the HSV high and low thresholds will be printed in real-time. When [OK !!!] appears below the box, it indicates successful recognition.

(3) Click the [learning_follow] button, and real-time tracking can be achieved as long as the contour of the learned color can be detected.

(4) Click the [follow_cancel] button to cancel tracking without exiting the program. Click the Exit button to exit the program.

Code Analysis

- Get HSV range of a certain area

```
# Convert color image to HSV
HSV = cv.cvtColor(img, cv.COLOR_BGR2HSV)
# Draw rectangle
cv.rectangle(img, (290, 190), (350, 250), (0, 255, 0), 2)
# Extract H, S, V values of each row and column into containers
for i in range(290, 350):
    for j in range(190, 250):
        H.append(HSV[j, i][0])
        S.append(HSV[j, i][1])
        V.append(HSV[j, i][2])
```

- Calculate the maximum and minimum of H, S, V separately

```
H_min = min(H);H_max = max(H)
S_min = min(S);S_max = max(S)
V_min = min(V);V_max = max(V)
```

- HSV range adjustment, the HSV values of blocks at different positions in the image have certain differences, so appropriate adjustment is needed.

```
if H_max + 2 > 255:H_max = 255
else:H_max += 2
if H_min - 2 < 0:H_min = 0
else:H_min -= 2
if S_min-10<0:S_min=0
else:S_min -= 15
if V_min-10<0:V_min=0
else:V_min -= 15
S_max = 255; V_max = 255
```

- Format conversion, convert the calculated data to int format

```
hsv_range = ((int(H_min), int(S_min), int(V_min)),(int(H_max), int(S_max),
int(V_max)))
```

The following contour detection, PID control and tracking are the same as color tracking.